Veeva Integrated curriculum Task – 1

E-Commerce Product Management System Using Core Java and OOP Principles

1. Project Overview

Develop a simple yet realistic **E-Commerce Product Management System** using Core Java and Object-Oriented Programming (OOP) principles.

The objective of this project is to simulate the essential functionalities of an e-commerce platform, including customer management, product catalog management, customer authentication, shopping cart operations, order processing, payment methods, order tracking, subscriptions, and discount coupons.

The system should demonstrate the practical application of **encapsulation, abstraction, inheritance, and polymorphism**, along with other core Java concepts and appropriate data management techniques.

2. Functional Requirements

2.1. Customer Management

The system should represent customers and securely manage their personal information, including:

- Name
- Email address
- Residential address
- Phone number

The customer class should follow the principles of encapsulation, abstraction, and data hiding. Sensitive customer information must not be accessed or modified directly from outside the class. Access to private data should be controlled through appropriate methods.

2.2. Customer Database Management

The system should maintain a collection of customer records that acts as an in-memory database.

It should support the following operations:

1. **Add Customer:** Register a new customer and store their information.
2. **Delete Customer:** Remove an existing customer's record.
3. **Update Customer:** Modify the personal information of an existing customer.
4. **Retrieve Customer by Name:** Return the corresponding customer record when a customer's name is provided.
5. **Retrieve Customers by City:** Return a list of customers whose residential address belongs to the specified city.
6. **Retrieve Customer by Phone Number:** Find the customer associated with the given phone number.
7. **Retrieve Customer by Email:** Find the customer associated with the given email address.

The system should validate customer information and handle cases where a customer record does not exist.

2.3. Product Catalog Management

The system should maintain a catalog of products available for purchase. Each product should contain relevant information, such as:

- Product ID
- Product name
- Price
- Company or brand name
- Product description
- Available stock quantity

The system should support adding new products, deleting existing products, updating product information, and retrieving products based on different criteria.

The following product search and retrieval operations must be supported:

1. **Search by Product ID:** Return the details of the product associated with the specified product ID.
2. **Search by Product Name:** Return a list of products matching the specified product name.
3. **Search by Product Name and Price:** Return products matching both the specified name and exact price.
4. **Search by Product Name and Price Range:** Return products matching the specified name whose prices fall within the given minimum and maximum price range.
5. **Search by Product Name and Company Name:** Return products matching the specified name and manufactured or supplied by the specified company.
6. **Search by Product Name, Company Name, and Price:** Return products matching all three specified criteria.

The catalog should support dynamic modifications so that new products can be added and existing product details or stock quantities can be updated whenever required.

2.4. Customer Registration and Login

The system should allow customers to register for an account and log in whenever they want to browse products or place orders.

The following functionalities should be provided:

- Customer registration with the required personal information and login credentials.
- Validation of registration details.
- Secure handling of passwords.
- Authentication of customers during login.
- Access to product browsing, shopping cart, and ordering functionalities only after successful authentication.

Each registered customer should have a unique identity within the system.

2.5. Shopping Cart Management

Customers should be able to add products to their shopping carts while exploring the product catalog.

The shopping cart should support the following operations:

1. Add a product to the cart.
2. Remove a product from the cart.
3. Update the quantity of a product in the cart.
4. View all products currently in the cart.
5. Calculate the subtotal of the cart based on product prices and quantities.
6. Calculate the applicable shipping cost.
7. Calculate the final order total before checkout.

The system should validate product availability and ensure that the requested quantities do not exceed the available stock.

2.6. Order Management and Processing

When a customer confirms the items in their shopping cart and proceeds to checkout, the system should create an order and store its details in the database.

Each order should contain information such as:

- Unique order ID
- Customer details or customer ID
- List of ordered products
- Product quantities and prices
- Order subtotal
- Shipping cost
- Applicable discounts
- Final order total
- Payment method and payment status
- Order creation date and time
- Current order status

The system should calculate the order amount based on the prices and quantities of the selected products.

The final order total should account for the shipping cost and any applicable discounts.

Order Total = Product Subtotal + Shipping Cost − Applicable Discounts

The system should also update the available stock after a successful order confirmation, subject to appropriate order and payment rules.

2.7. Order Status Management

The system should allow an administrator to update an order's status as it progresses through the delivery process.

The following order statuses should be supported:

1. **Order Placed:** The order has been successfully created and received by the system.
2. **Order Dispatched:** The order has been handed over for shipment.
3. **In Transit:** The order is being transported through the courier network.
4. **Out for Delivery:** The courier agent is delivering the order to the customer.
5. **Delivered:** The order has been successfully delivered to the customer.
6. **Return Requested:** The customer has requested a return.
7. **Returned and Received:** The returned product has been received by the seller or warehouse.

The system should maintain the current status of each order and allow customers to view the status of their orders.

Appropriate status transitions should be enforced to prevent invalid order updates.

2.8. Order Tracking

Customers should be able to view tracking information for their orders once the orders have been dispatched.

The tracking details should include, where applicable:

- Order ID
- Tracking ID
- Courier company name
- Current shipment status
- Dispatch date
- Latest tracking update

As the administrator updates the order status, the customer should be able to view the latest available tracking information.

Since this project uses Core Java, shipment tracking can initially be simulated within the application without integrating an external courier service.

2.9. Payment Management

The system should support multiple payment methods through a common payment-processing mechanism.

The following payment methods should be implemented:

1. **UPI:** Simulate payment through a UPI-based method.
2. **Debit Card:** Simulate payment using a debit card.
3. **Credit Card:** Simulate payment using a credit card.
4. **Cash on Delivery (COD):** Allow customers to pay when the order is delivered.
5. **Gift Card:** Allow customers to redeem a gift card to pay for eligible orders.

Each payment method should have its own payment-processing behavior while following a common abstraction.

The system should maintain payment details such as the payment method, payment amount, transaction reference, and payment status.

Payment processing should be simulated for this project. Actual banking, UPI, or card transactions are not required.

2.10. Shipping Cost Calculation

The system should calculate the final order total by combining the product subtotal with the applicable shipping cost and subtracting any eligible discounts.

The shipping cost should be calculated separately so that different shipping policies can be introduced in the future.

For example:

Final Order Total = Product Subtotal + Shipping Cost − Discounts

The shipping cost may depend on factors such as the order value, customer subscription status, or predefined shipping rules.

Customers with an active eligible subscription should receive free shipping according to the subscription policy.

2.11. Gift Card Payment

After implementing the basic payment methods, extend the system to support gift card payments.

The gift card functionality should include:

- Unique gift card number or code.
- Gift card balance.
- Gift card validity or expiration date, if applicable.
- Validation of the gift card before redemption.
- Deduction of the applicable amount from the gift card balance.
- Prevention of redemption when the gift card is invalid, expired, or has insufficient balance.

If the gift card balance is insufficient to cover the order total, the system may allow the customer to pay the remaining amount using another supported payment method.

The system should update the gift card balance after successful redemption.

2.12. Customer Subscription Management

As an optional enhancement, introduce a subscription feature that provides eligible customers with benefits such as free shipping.

The system should support the following subscription plans:

- Monthly
- Quarterly
- Half-yearly
- Yearly

Each subscription should contain details such as the subscription ID, customer ID, subscription plan, start date, expiration date, subscription fee, and current status.

The system should support the following operations:

1. Subscribe to a plan.
2. View available subscription plans.
3. View current subscription details.
4. Check whether a subscription is active or expired.
5. Renew or cancel a subscription, according to the defined rules.

Customers with an active subscription should not be charged shipping fees for eligible orders.

The system should calculate subscription validity based on the selected plan's duration.

2.13. Discount Coupons for Frequent Customers

As another optional enhancement, implement a discount coupon system to reward customers who place multiple orders or meet a predefined spending threshold.

The system should be able to evaluate a customer's order history and determine whether the customer qualifies for a discount coupon.

For example, a customer may receive a coupon after placing a specified number of orders or exceeding a predefined cumulative spending amount.

The coupon system should support:

- Unique coupon codes.
- Eligibility rules based on order count or cumulative order value.
- Discount percentage or fixed discount amount.
- Minimum order value, if applicable.
- Coupon validity period.
- Coupon redemption and usage tracking.

- Validation to prevent invalid or repeated redemption.

The system should apply eligible discounts during checkout and recalculate the final order total accordingly.

The rules for earning, issuing, and redeeming coupons should be configurable.

3. Object-Oriented Programming Requirements

The entire application should be designed using appropriate Object-Oriented Programming principles.

3.1. Encapsulation

Keep the data of classes private and provide controlled access through suitable public methods. Prevent unauthorized modification of customer, product, order, and payment information.

3.2. Abstraction

Expose only the necessary operations to users of a class while hiding internal implementation details. Use interfaces and abstract classes wherever they provide a meaningful abstraction.

3.3. Inheritance

Identify appropriate parent-child relationships between classes to reuse common properties and behaviors. For example, different payment types may share common payment-related attributes and operations.

3.4. Polymorphism

Use method overriding and, where appropriate, method overloading to support different behaviors through common interfaces or parent types.

For example, UPI, debit card, credit card, COD, and gift card payments should be processed according to their respective implementations through a common payment abstraction.

3.5. Additional Core Java Concepts

Use other relevant Core Java features wherever appropriate, including:

- Collections Framework for storing and managing customers, products, carts, orders, and coupons.
- Exception handling for invalid inputs, failed operations, unavailable products, and payment errors.
- Enums for order statuses, payment statuses, and subscription plans.
- Java Streams and lambda expressions for filtering and searching customers and products.
- Date and time API for order timestamps, subscription periods, and coupon validity.
- Appropriate validation techniques to maintain data consistency.

4. Data Storage Requirements

Initially, use Java collections such as ArrayList, HashMap, or other suitable data structures to simulate an in-memory database.

The system should support the addition, retrieval, updating, and deletion of records without requiring an external database.

Where appropriate, use unique identifiers to identify customers, products, orders, gift cards, and coupons.

The design should make it possible to replace the in-memory storage with a database such as MySQL or MongoDB in a future version without significantly changing the business logic.

5. Expected Outcome

The final application should simulate the essential workflow of a realistic e-commerce platform:

1. A customer registers and logs in.
2. The customer browses the product catalog and searches for products using different criteria.
3. The customer adds products to the shopping cart and modifies quantities as needed.
4. The system calculates the product subtotal, shipping cost, applicable discounts, and final order total.
5. The customer selects a payment method and confirms the order.
6. The system creates the order, records the payment status, and updates product stock according to the defined rules.
7. The administrator updates the order status as the order progresses through dispatch, transit, delivery, or return.
8. The customer views the order status and available tracking information.
9. Eligible customers receive free shipping through active subscriptions.
10. Frequent customers can qualify for discount coupons.
11. Customers can redeem gift cards as a payment method.

The application should be modular, maintainable, extensible, and easy to test. The primary objective is to understand how Core Java and OOP principles work together to build a realistic, object-oriented application.

The initial implementation should focus on the core customer, product, cart, order, and payment functionalities. Gift cards, subscriptions, and discount coupons can then be added incrementally as advanced features.